

MS&E 228: Applied Causal Inference Powered by ML and AI

Lecture 11: CATE Model Selection, Interpretation & Validation

Vasilis Syrgkanis

Stanford University

Winter 2026

Readings: *Applied Causal Inference Powered by ML and AI*, §15.

Where We Are

In the last lecture we saw four meta-learners for estimating the CATE:

- ▶ **S-Learner:** Single model for the outcome, derive CATE by differencing predictions.
- ▶ **T-Learner:** Separate models for treated and control, $\text{CATE} = \text{difference}$.
- ▶ **DR-Learner:** Generalizes the doubly robust estimator to CATE estimation.
- ▶ **R-Learner:** Generalizes the DML estimator to CATE estimation.

We focused on **linear** and **causal forest** variants, which produce confidence intervals for CATE predictions.

Today, we drop the CI requirement and allow any ML method in the final stage.

Goals for Today

1. **Model Selection:** How should we select among many CATE models (S, T, DR, $R \times$ various ML methods)?
2. **Interpretation:** How do we interpret a black-box CATE model? (Distillation trees, SHAP)
3. **Statistical Validation:** How do we test that the CATE has picked up real heterogeneity? Can we identify subgroups with statistically different effects? Is the CATE useful for treatment prioritization?
4. **In-Class Activity:** Apply these ideas to a real-world digital health paper.

Part I: CATE Model Selection

Choosing the Best Model from a Zoo of Learners

The Model Selection Problem

Each meta-learner (S, T, DR, R) follows the same recipe:

1. Fit **nuisance models** in the first stage (outcome regression $\hat{g}$, propensity $\hat{e}$).
2. Construct an appropriate **loss function** (regression or weighted regression) using these nuisance models.
3. Use a **generic ML method** to fit the final-stage regression $\implies$ CATE estimate.

So we face a **combinatorial selection problem**:

- ▶ **Which meta-learner?** S, T, DR, R (also X-learner; see book)
- ▶ **Which final-stage ML method?** XGBoost, Random Forest, Neural Net, Lasso, ElasticNet, ...
- ▶ **Which hyperparameters?** Depth, learning rate, regularization, ...

How do we navigate this?

Step 1: Hyperparameter Tuning Within Each Learner

Key insight: Each meta-learner is a **loss minimizer**. Use the corresponding out-of-sample loss for hyperparameter tuning within each method.

Practical Recipe

- ▶ **DR-Learner** → Use the **DR-Loss** (MSE of predicting DR pseudo-outcomes).
This is just a standard MSE metric. Use FLAML or any AutoML package.
- ▶ **R-Learner** → Use the **R-Loss** (sample-weighted MSE).
Caution: Don't normalize the weights! You may need a custom weighted MSE metric. (See Lecture 8 notes for the subtlety)
- ▶ **S-Learner, T-Learner** → Use their native outcome prediction losses.

Step 1: FLAML Example for DR-Learner

```
from flaml import AutoML

# DR pseudo-outcomes computed from first-stage nuisance models
# Y_dr = Y_pseudo_outcomes (computed on training data)
# X = features

automl = AutoML()
automl.fit(
    X_train, Y_dr_train,
    task="regression",
    metric="mse",          # DR-Loss = MSE of predicting Y_dr
    time_budget=120,      # seconds
    estimator_list=["xgboost", "rf", "lgbm", "extra_tree"],
)

# Best model is the one that minimizes out-of-sample DR-Loss
best_cate_dr = automl.model
```

Step 1: FLAML Example for R-Learner

For the R-Learner, replace `metric="mse"` with a **custom weighted MSE** that does not normalize the weights. Important when using cross validation ('cv') for model selection.

```
from flaml import AutoML

def weighted_sse(X_val, y_val, estimator, labels,
                 X_train, y_train, weight_val=None, weight_train=None, *args):
    """Un-normalized weighted sum of squared errors on validation set."""
    y_pred = estimator.predict(X_val)
    loss = np.mean(weight_val * (y_val - y_pred) ** 2)
    return loss, {"weighted_sse": loss}

automl = AutoML(task="regression", eval_method='cv',
                 metric=weighted_sse, # custom weighted metric (un-normalized)
                 time_budget=60)
automl.fit(X_train=X, y_train=Y_res / D_res, sample_weight=D_res**2)
```


Step 2: Select Best Model via DR-Loss

After tuning each learner, we have a zoo of candidate models $\mathcal{M} = \{\hat{\tau}_1, \hat{\tau}_2, \dots, \hat{\tau}_M\}$. How do we pick the best one?

The DR-Loss as a Universal Scoring Rule

Use **separate scoring sample** and compute DR-Loss for *all* candidate CATE models on that sample. Choose model with the smallest score-sample DR-Loss.

$$\hat{\tau}_{\text{best}} = \arg \min_{\hat{\tau} \in \mathcal{M}} \frac{1}{n_{\text{score}}} \sum_{i \in \text{ScoreSet}} \left(Y_i^{DR} - \hat{\tau}(X_i) \right)^2$$

For the nuisance models needed to compute DR pseudo-outcomes on the scoring sample, use all of the training data. If you have lots of data, you can also cross-fit within the scoring data itself.

Step 2: Select Best Model — Pseudocode

```
# Assume:  
# - candidate_models = a list of fitted CATE models  
# - X_score, Y_score, D_score = scoring data  
# - g_hat, e_hat = nuisance models trained on main training data  
  
Y_dr_score = compute_dr_pseudooutcomes(Y_score, D_score, g_hat, e_hat)  
  
best_model = None  
min_dr_loss = float('inf')  
  
for model in candidate_models:  
    tau_hat_score = model.predict(X_score)  
    dr_loss = np.mean((Y_dr_score - tau_hat_score)**2)  
  
    if dr_loss < min_dr_loss:  
        min_dr_loss = dr_loss  
        best_model = model  
  
# best_model is your final CATE estimator
```

Why the DR-Loss Is Special

Key Theoretical Result (Randomized Experiments)

In a randomized experiment (where the propensity is known), the DR-Loss is a **perfect out-of-sample loss**:

$$\mathbb{E}\left[(Y_i^{DR} - \hat{\tau}(X_i))^2\right] = \mathbb{E}\left[(\tau(X_i) - \hat{\tau}(X_i))^2\right] + \text{const}$$

It minimizes the RMSE with respect to the **individual treatment effect**—without any assumptions on nuisance models. It can't get better than that!

So in an RCT, just use a constant propensity model and the DR-Loss gives you the oracle model selector.

Technical Note: Observational Studies

In observational settings, the DR-Loss is still a good selector, but with a caveat:

Finite-Sample Guarantee

With high probability, the MSE of the selected model satisfies:

$$\text{MSE}(\hat{\tau}_{\text{chosen}}) \lesssim \text{MSE}(\hat{\tau}_{\text{best}}) + \sqrt{\frac{\log M}{n}} + \text{RMSE}(\hat{g}) \cdot \text{RMSE}(\hat{\rho})$$

where M = number of candidate models, n = scoring sample size.

Ideal bound: $\text{RMSE}(\hat{\tau}_{\text{chosen}}) \lesssim \text{RMSE}(\hat{\tau}_{\text{best}}) + \sqrt{\log M/n} + \text{RMSE}(\hat{g}) \cdot \text{RMSE}(\hat{\rho})$.

This is achievable under a slightly stronger assumption: bounded *fourth-moment* error of nuisances, i.e., $(\mathbb{E}[(Y - g(X))^4])^{1/4} \leq \epsilon$.

Bottomline: If nuisance models are reasonably good, the DR-Loss is an excellent model selector. See §15.2 of the CausalML book for details.

Alternative: Stacking CATE Models

Instead of picking one best model, we can **stack** them: create an ensemble where the final prediction is a weighted average of all candidate model predictions.

Stacking via DR-Loss Minimization

Find weights $w = (w_1, \dots, w_M)$ that minimize the DR-Loss of the stacked model

$$\hat{\tau}_{\text{stack}}(x) = \sum_{m=1}^M w_m \hat{\tau}_m(x)$$

on the scoring set.

$$\hat{w} = \arg \min_{w \in W} \frac{1}{n_{\text{score}}} \sum_{i \in \text{ScoreSet}} \left(Y_i^{\text{DR}} - \sum_{m=1}^M w_m \hat{\tau}_m(X_i) \right)^2$$

where W is some space (e.g., the simplex for interpretability, or any set of weights for flexibility).

Stacking: Pseudocode

```
from sklearn.linear_model import LinearRegression

# Predictions of each candidate model on the scoring set
# Shape: (n_samples, n_models)
all_preds_score = np.array(
    [model.predict(X_score) for model in candidate_models]).T

# Target: DR pseudo-outcomes on the scoring set
Y_dr_score = compute_dr_pseudooutcomes(Y_score, D_score, g_hat, e_hat)

# Fit OLS to find the weights
stacker = LinearRegression(fit_intercept=False)
stacker.fit(all_preds_score, Y_dr_score)

# The final stacked model's prediction is a weighted average
final_cate_pred = stacker.predict(X_score)
```

The Complete Data Splitting Strategy

Recommended Data Split: 60 / 20 / 20

1. **Training set (60%):** Train all meta-learners (nuisance models + final CATE models). Tune hyperparameters within each learner using its native loss.
2. **Scoring set (20%):** Compute DR pseudo-outcomes using nuisance models from training. Select or stack CATE models via the DR-Loss.
3. **Validation set (20%):** Reserved for statistical tests of the chosen CATE model. *Never touched during training or selection.*

We now have a “best guess” CATE model. Next: how do we interpret it, and how do we validate it?

Part II: CATE Interpretation

Making Black-Box Models Understandable

Running Example: The Welfare Experiment

Throughout the rest of this lecture, we use the **Welfare Experiment** from Chapter 15 of the CausalML book (also from your earlier homework).

Experiment Setup

- ▶ **Treatment ($D=1$):** Survey asks about willingness to support a “Welfare Program.”
- ▶ **Control ($D=0$):** Same survey but calls it “Assistance to the Poor.”
- ▶ **Outcome:** Willingness to support (binary).
- ▶ **Data:** $n = 12,907$ individuals, 42 covariates (GSS data).
- ▶ **Goal:** Estimate the CATE of the word “welfare” on support. Expected to be negative and heterogeneous.

We assume we have already trained and selected a final CATE model $\hat{\tau}(X)$ using the pipeline from Part I.

Interpretation Tool 1: Distillation Trees

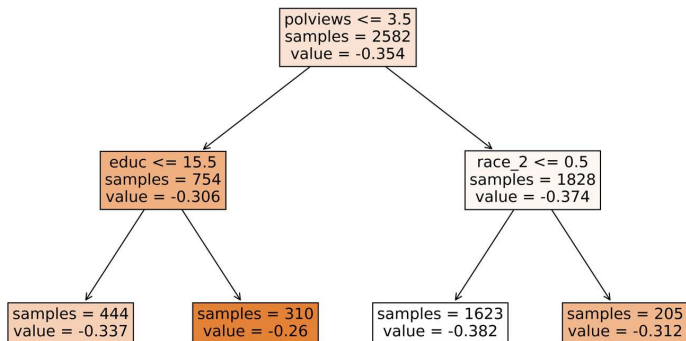
A simple and powerful way to interpret a complex CATE model: **distill** its predictions into a shallow, interpretable decision tree.

Algorithm: Distillation Tree

1. Compute CATE predictions: $\hat{\tau}(X_i)$ for all training samples.
2. Fit a shallow regression tree on these predictions:
`DecisionTreeRegressor(max_depth=2).fit(X, CATE_predictions)`
3. Visualize the tree to see the most important interactions.

The tree identifies the **dominant features and thresholds** that drive heterogeneity. It won't capture everything, but it gives you the “big picture.”

Welfare Experiment: CATE Model Distillation Tree



Interpretation: The model learned: Right-leaning, white individuals have the *most negative* reaction to the word “welfare.” Left-leaning, well-educated individuals have the *least negative* reaction to the word “welfare.”

EconML: SingleTreeCateInterpreter

EconML provides a built-in tool to distill any CATE model into an interpretable tree:

```
from econml.cate_interpreter import SingleTreeCateInterpreter

# Fit a shallow tree to explain the CATE model
intrp = SingleTreeCateInterpreter(
    include_model_uncertainty=True,
    max_depth=2, min_samples_leaf=10)
intrp.interpret(est, X) # est is any fitted CATE model

# Visualize the tree inline
plt.figure(figsize=(25, 5))
intrp.plot(feature_names=feature_names, fontsize=12)
plt.show()
```

`include_model_uncertainty=True` incorporates the model's own uncertainty into the tree splits, providing more robust interpretations.

Interpretation Tool 2: SHAP Values

For a more nuanced, **feature-level** understanding, apply **SHAP (SHapley Additive exPlanations)** to the CATE model.

SHAP decomposes each individual CATE prediction into the sum of contributions from each feature:

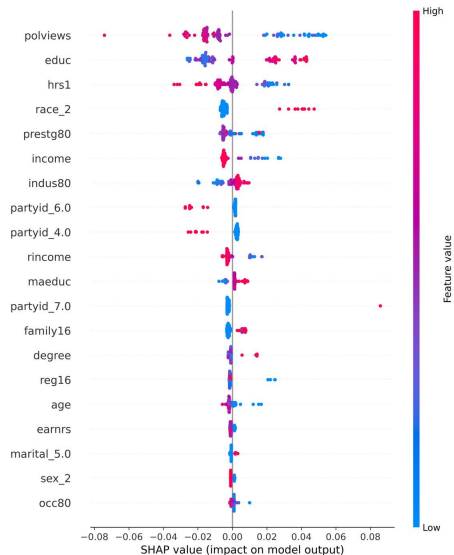
$$\hat{\tau}(x_i) = \phi_0 + \sum_{j=1}^p \phi_j(x_i)$$

where $\phi_j(x_i)$ is the contribution of feature j for individual i .

What you get:

- ▶ **Feature importance:** Which features contribute most to heterogeneity?
- ▶ **Directionality:** In which direction does each feature push the CATE?
- ▶ **Per-individual explanations:** Why does this person have a high/low CATE?

Welfare Experiment: CATE Model SHAP Values



Distillation trees give you the “big picture” interactions, while SHAP gives you the granular, per-feature story. Use both!

EconML: SHAP for CATE Models

EconML integrates directly with the SHAP library to explain CATE predictions:

```
import shap

# Compute SHAP values for the CATE model; est is an econml estimator
shap_values = est.shap_values(
    X[:100],
    feature_names=feature_names,
    background_samples=100)

# Summary plot: feature importance + directionality
shap.summary_plot(shap_values['Y0']['T0_1'])
```

Note: The shap_values output is a nested dict indexed by outcome (Y0) and treatment contrast (T0_1). For binary treatment, T0_1 gives the SHAP values for the effect of $D = 1$ vs. $D = 0$.

Part III: Statistical Validation

Testing Whether Heterogeneity Is Real

The Validation Framework

We have selected our best CATE model $\hat{\tau}(X)$. Now we need to **statistically validate** it on the held-out **validation set**.

Setup:

- ▶ Compute DR pseudo-outcomes Y^{DR} on the validation set using nuisance models trained on the training set.
- ▶ All tests below use only the validation data and the fitted $\hat{\tau}$.

We will perform three complementary tests:

1. **Best Linear Predictor (BLP) Test:** Does $\hat{\tau}$ carry *any* signal of heterogeneity?
2. **Group Average Treatment Effects (GATEs):** Can $\hat{\tau}$ identify groups with statistically different effects?
3. **Qini Curve:** Is $\hat{\tau}$ useful for treatment prioritization?

Test 1: Best Linear Predictor (BLP)

The simplest sanity check. Regress the DR pseudo-outcomes on the **demeaned** CATE predictions:

The BLP Test

$$\text{OLS: } Y_i^{DR} = \alpha + \beta (\hat{\tau}(X_i) - \bar{\hat{\tau}}) + \epsilon_i$$

Interpreting the coefficients:

- ▶ $\hat{\alpha}$: Estimate of the ATE.
- ▶ $\hat{\beta}$: Estimates $\frac{\text{Cov}(Y(1) - Y(0), \hat{\tau}(X))}{\text{Var}(\hat{\tau}(X))}$. This is the **beta-coefficient** between the true ITE and your CATE model.

Interpreting the BLP Coefficient $\hat{\beta}$

What should we hope for?

- ▶ If $\hat{\tau}$ were the *true* CATE, then $\beta = 1$.
- ▶ If $\hat{\tau}$ carries *any* signal of heterogeneity: β should be **statistically non-zero**.
- ▶ If $\hat{\tau}$ is garbage: $\beta \approx 0$ (no correlation with true ITE; or no effect heterogeneity).

Decision rule:

1. Check if the 95% CI for $\hat{\beta}$ **contains 1** $\implies$ fantastic! Well-calibrated model.
2. Check if the 95% CI for $\hat{\beta}$ **excludes 0** $\implies$ cool! Heterogeneity detected.
3. If the CI contains 0 $\implies$ your model has not picked up meaningful heterogeneity.

Welfare Experiment: BLP Test

	coef	std err	P > z	[0.025	0.975]
const	-0.3839	0.016	0.000	-0.416	-0.352
$\tau_*(X)$	1.4655	0.267	0.000	0.943	1.988

Result: $\hat{\beta} = 1.47$ with 95% CI [0.94, 1.99].

- ▶ CI excludes 0 $\implies$ **heterogeneity detected.**
- ▶ CI contains 1 $\implies$ model is **well-calibrated!**

BLP Test: Pseudocode

```
import statsmodels.api as sm

# Compute DR pseudo-outcomes on validation set
Y_dr_val = compute_dr_pseudoutcomes(
    Y_val, D_val, g_hat, e_hat)

# Demean CATE predictions
tau_hat_val = cate_model.predict(X_val)
tau_demeaned = tau_hat_val - np.mean(tau_hat_val)

# Run OLS: Y_dr ~ intercept + tau_demeaned
X_ols = sm.add_constant(tau_demeaned)
results = sm.OLS(Y_dr_val, X_ols).fit(cov_type='HCO')

# Check the beta coefficient
print(results.summary())
# alpha = ATE estimate, beta = heterogeneity signal
# Want: beta CI excludes 0 (signal); contains 1 (calibrated)
```

Test 2: Group Average Treatment Effects (GATEs)

A second, more granular check: can $\hat{\tau}$ identify groups with **statistically different** treatment effects?

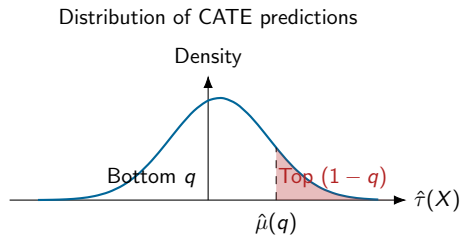
Idea: Look at the groups your CATE model *believes* have high vs. low effects, and validate that belief on the held-out validation data.

Test 2: Group Average Treatment Effects (GATEs)

Step 1: Compute the distribution of CATE predictions on the *training* data.

Let $\hat{\mu}(q)$ be the empirical q -th percentile

($\approx q \cdot 100\%$ of units have CATE below $\hat{\mu}(q)$)



Step 2: Define groups based on CATE quantiles:

$$G_0 = \{i : \hat{\tau}(X_i) < \hat{\mu}(0.25)\} \quad (\text{bottom 25\%: "non-responders"})$$

$$G_1 = \{i : \hat{\tau}(X_i) > \hat{\mu}(0.75)\} \quad (\text{top 25\%: "high responders"})$$

GATEs: Estimating Group Effects

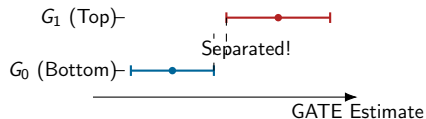
Step 3: For each group, estimate the GATE using the DR estimator on **validation set**:

GATE Confidence Intervals

$$\widehat{\text{GATE}}(G_k) = \frac{1}{|G_k|} \sum_{i \in G_k} Y_i^{DR}$$

$$95\% \text{ CI} = \widehat{\text{GATE}}(G_k) \pm 1.96 \sqrt{\frac{\widehat{\text{Var}}_{G_k}(Y^{DR})}{|G_k|}}$$

What we hope for: The bottom group's CI should lie *below* the top group's CI.
If that happens: **success!**



At that point, pause and celebrate for a while!

GATEs: Finer Groups and Calibration

More ambitiously, define **quartile groups** (or even finer):

$$G_0 = \{i : \hat{\tau}(X_i) < \hat{\mu}(0.25)\}$$

$$G_1 = \{i : \hat{\mu}(0.25) \leq \hat{\tau}(X_i) < \hat{\mu}(0.50)\}$$

$$G_2 = \{i : \hat{\mu}(0.50) \leq \hat{\tau}(X_i) < \hat{\mu}(0.75)\}$$

$$G_3 = \{i : \hat{\tau}(X_i) \geq \hat{\mu}(0.75)\}$$

Compute the GATE and 95% CI for each group. Ideally, the GATEs should be **monotonically ordered** (increasing from G_0 to G_3).

You can go finer than quartiles if you have enough samples, but be careful about statistical power.

The Calibration Plot

Visualize the GATEs with a **calibration plot**:

- **X-axis:** What the CATE model *believes* is the group average effect, i.e.,

$$\bar{\hat{\tau}}_{G_k} = \frac{1}{|G_k|} \sum_{i \in G_k} \hat{\tau}(X_i).$$

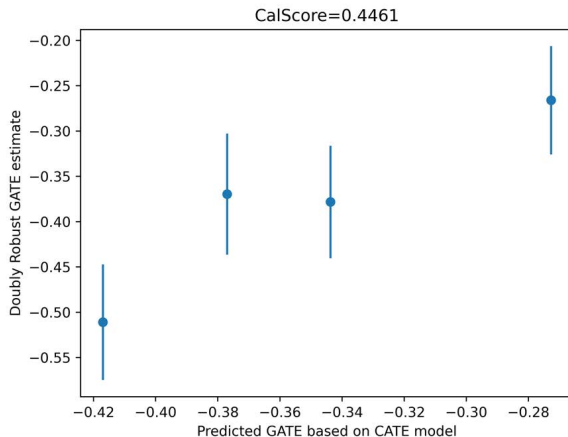
- **Y-axis:** The DR point estimate of the GATE on the validation set, with 95% CI.

Ideal outcome: Points lie close to the **45-degree line**—what the model believes matches what the data shows. If so, the CATE model is **well-calibrated**.

You can also compute a **calibration error**:

$$\text{CalError} = \sum_k \frac{|G_k|}{n} \cdot |\widehat{\text{GATE}}(G_k) - \bar{\hat{\tau}}_{G_k}|$$

Welfare Experiment: CATE Model Calibration Plot



Bottom quartile (predicted ≈ -0.42 , DR estimate ≈ -0.51) and top quartile (predicted ≈ -0.28 , DR estimate ≈ -0.26) are **statistically separated**.

Characterizing the Groups

Once you've found statistically different groups, you want to **understand what makes them different**.

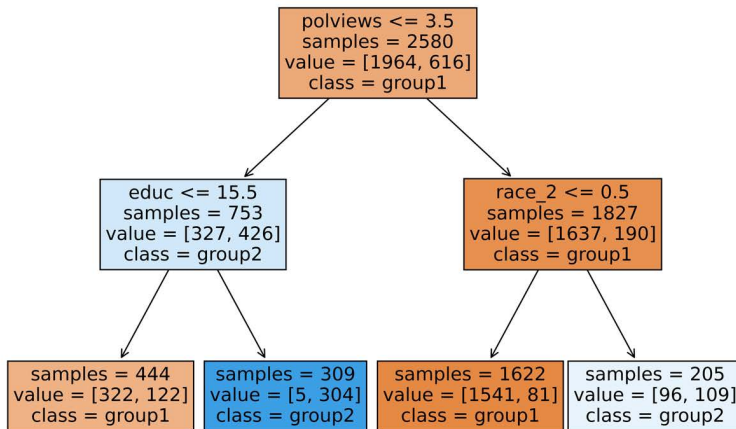
Two approaches:

1. **Difference in means:** For each feature X_j , compute $\bar{X}_j^{G_1} - \bar{X}_j^{G_0}$. This tells you how the high-responder and low-responder groups differ on each feature.
2. **Classification tree:** Build a dataset with samples from G_0 (label 0) and G_1 (label 1). Train a shallow decision tree classifier on features X . This gives a tree-based characterization of the dominant factors.

Welfare Experiment: Low vs High Responder Group Characterization

	group1 mean $\pm$ s.e.	group2 mean $\pm$ s.e.	group1 - group2 mean $\pm$ s.e.
hrs1	44.16 $\pm$ 0.30	36.74 $\pm$ 0.58	7.42 $\pm$ 0.88
income	11.52 $\pm$ 0.03	10.61 $\pm$ 0.10	0.92 $\pm$ 0.12
rincome	10.58 $\pm$ 0.05	9.23 $\pm$ 0.14	1.35 $\pm$ 0.19
age	41.75 $\pm$ 0.27	37.52 $\pm$ 0.49	4.23 $\pm$ 0.76
polviews	4.39 $\pm$ 0.03	3.07 $\pm$ 0.05	1.32 $\pm$ 0.08
educ	13.58 $\pm$ 0.06	15.41 $\pm$ 0.11	-1.82 $\pm$ 0.17
earnrs	1.81 $\pm$ 0.02	1.60 $\pm$ 0.03	0.21 $\pm$ 0.05
sibs	3.40 $\pm$ 0.06	3.42 $\pm$ 0.14	-0.02 $\pm$ 0.20
childs	1.68 $\pm$ 0.03	1.24 $\pm$ 0.06	0.44 $\pm$ 0.09
occ80	351.52 $\pm$ 5.66	280.24 $\pm$ 8.53	71.28 $\pm$ 14.19

Welfare Experiment: Low vs High Responder Group Characterization



Test 3: The Qini Curve

Question: Is the CATE model useful for **treatment prioritization**?

Suppose you want to treat a fraction $\approx (1 - q)$ of the population. If you believe your CATE model, you should treat the group:

$$G(q) = \{i : \hat{\tau}(X_i) > \hat{\mu}(q)\}$$

where $\hat{\mu}(q)$ is the empirical q -th percentile of CATE predictions.

Benefit of CATE-based prioritization:

$$\text{Benefit-Priority}(q) = \mathbb{E}[Y(1) - Y(0) \mid \hat{\tau}(X) > \hat{\mu}(q)] \cdot \Pr(\hat{\tau}(X) > \hat{\mu}(q))$$

Benefit of random treatment (same fraction):

$$\text{Benefit-Random}(q) = \text{ATE} \cdot \Pr(\hat{\tau}(X) > \hat{\mu}(q))$$

The Qini Curve: Definition

The **Qini curve** at q measures the *gain* from CATE-based targeting over random targeting:

Qini Curve

$$\begin{aligned}\tau_{\text{Qini}}(q) &= \Pr(\hat{\tau}(X) > \hat{\mu}(q)) \cdot (\text{GATE}(G(q)) - \text{ATE}) \\ &= \text{Cov}(Y(1) - Y(0), \mathbf{1}\{\hat{\tau}(X) > \hat{\mu}(q)\})\end{aligned}$$

Key insight: The Qini value at q equals the **covariance** of the true ITE with the indicator of your model believing the CATE is above the q -th percentile.

This can be identified by the DR analogue:

$$\tau_{\text{Qini}}(q) = \text{Cov}(Y^{DR}, \mathbf{1}\{\hat{\tau}(X) > \hat{\mu}(q)\})$$

Estimating the Qini Curve

We estimate the Qini curve using the **empirical covariance** on the validation set:

Qini Estimator and Standard Error

$$\hat{\tau}_{\text{Qini}}(q) = \widehat{\text{Cov}}_n(Y^{DR}, \mathbf{1}\{\hat{\tau}(X) > \hat{\mu}(q)\})$$

Standard error:

$$\text{SE}(q) = \sqrt{\frac{\widehat{\text{Var}}_n[(Y^{DR} - \bar{Y}^{DR})(\mathbf{1}\{\hat{\tau}(X) > \hat{\mu}(q)\} - (1 - \hat{q}))]}{n}}$$

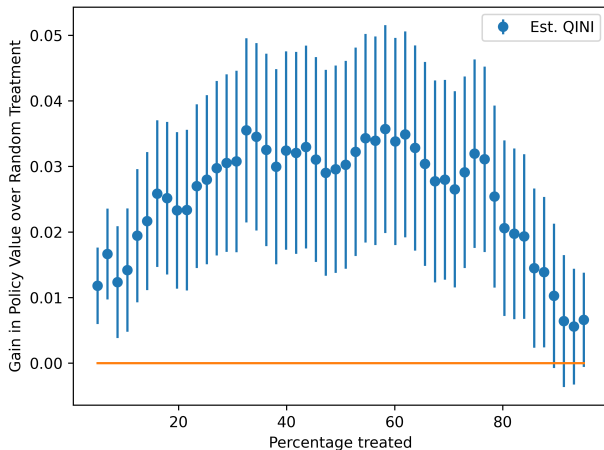
where $1 - \hat{q}$ is the empirical fraction treated on the validation set.

The Qini curve plots:

- ▶ **X-axis:** Fraction treated ($1 - \hat{q}$).
- ▶ **Y-axis:** Qini value $\hat{\tau}_{\text{Qini}}(q)$ with 95% confidence bands.

If the curve is above zero, your CATE model is better than random for prioritization.

Welfare Experiment: Qini Curve



The Qini curve is **above zero** even with confidence bands $\implies$ the CATE model provides genuine value for targeting.

Qini Curve: Pseudocode

```
q_grid = np.linspace(0.05, 0.95, 50)
cutoffs = np.percentile(tau_hat_train, q_grid * 100)

qini, se, fraction_treated = np.zeros(50), np.zeros(50), np.zeros(50)
for i, c in enumerate(cutoffs):
    ind = (tau_hat_val > c).astype(float)
    fraction_treated[i] = np.mean(ind)

    # Covariance of Y_dr with the indicator
    AB = (Y_dr_val - Y_dr_val.mean()) * (ind - fraction_treated[i])
    qini[i] = np.mean(AB)
    se[i] = np.std(AB) / np.sqrt(len(Y_dr_val))

# Plot: fraction treated vs. Qini value with standard errors
plt.fill_between(fraction_treated, qini - 1.96*se, qini + 1.96*se, alpha=0.3)
plt.plot(frac, qini); plt.axhline(y=0, color='red', ls='--')
```

EconML: DRTester — All-in-One Validation

EconML's DRTester automates the entire validation pipeline (BLP, GATEs, Qini, TOC) in a few lines. See Example Notebook.

```
from econml.validate.drtester import DRTester

# Step 1: Initialize with nuisance models and CATE model
tester = DRTester(
    model_regression=GradientBoostingRegressor(),
    model_propensity=RandomForestClassifier(),
    cate=est  # any fitted CATE estimator
)
# Step 2: Fit nuisance models on train, evaluate on val
tester.fit_nuisance(Xval, Dval, Yval, Xtrain, Dtrain, Ytrain)
# Step 3: Run all validation tests at once
res = tester.evaluate_all(Xval, Xtrain)
# Step 4: View the summary table
res.summary()
```

DRTester: Summary Output and Plots

The `summary()` method returns a DataFrame with all key metrics:

Treatment	BLP Est	BLP p-val	Qini Est	Qini p-val	AUTO C Est	Cal R^2
$D = 1$ (no het.)	0.044	0.854	-0.009	0.352	-0.011	-9.92
$D = 2$ (het.)	1.032	0.000	0.373	0.000	1.049	-0.09

```
# Calibration plot for treatment 1
res.plot_cal(tmt=1)

# Qini curve with uniform confidence bands
res.plot_qini(tmt=1, err_type='ucb2')

# TOC (Targeting Operating Characteristic) curve
res.plot_toc(tmt=1)
```

The `err_type` parameter controls confidence bands: 'pointwise' (default), 'ucb1' (one-sided), or 'ucb2' (two-sided uniform).

Part IV: In-Class Activity

Applying CATE Analysis to a Real-World Problem

Case Study: Digital Health Interventions for Youth with Diabetes

Scenario

A children's hospital uses a **remote patient monitoring (RPM)** app to help youth with type 1 diabetes manage their blood glucose. Clinicians can send personalized **digital health interventions (DHIs)** through the app—motivational messages, reminders, educational tips, and behavioral nudges.

The hospital wants to know: **which patients benefit most from these interventions, and when?**

The Data

You have access to a dataset with the following structure for each patient on each day:

- ▶ **Patient Features:** Demographics, clinical history, baseline glucose control (e.g., avg glucose, time in range, continuous glucose monitoring time-series signal for the weeks prior to the intervention), engagement with the app, time since diagnosis, etc.
- ▶ **Intervention:** A binary indicator of whether a clinician sent a message to the patient on that day.
- ▶ **Message Content:** The raw text of the message that was sent (if any).
- ▶ **Outcome:** A measure of glycemic control (e.g., time-in-range for blood glucose) in the days *following* the intervention.

Key challenge: This is an **observational study**—clinicians decide when and whom to message based on their clinical judgment. The treatment is *not* randomly assigned!

Your Task (In Groups, 15 min)

1. Frame the HTE problem:

- ▶ What is the treatment D ? What is the outcome Y ?
- ▶ What features X might drive heterogeneity?
- ▶ What is $\tau(x)$ in this context?
- ▶ How should you handle the **message content**? Is a binary D enough? How could you create a richer representation of the treatment? (Think about NLP.)

2. Devise a solution plan:

- ▶ Which meta-learners would you use (S , T , DR , R)?
- ▶ How would you do model selection? (DR-Loss on a scoring set?)
- ▶ What validation tests would you run?

3. Discuss interpretation and deployment:

- ▶ How would you explain your CATE findings to the clinicians?
- ▶ What tools from today's lecture would be most useful?

Share-Out and Discussion

Key questions for the share-out:

- ▶ How did you handle the fact that this is an *observational* setting? What are the implications for the DR-Loss guarantee?
- ▶ How did you propose to represent the message content? Raw embeddings vs. clinician-defined categories? What are the pros and cons?
- ▶ Which validation test would be most convincing to a clinician? (Probably the calibration plot and GATEs—showing that high-predicted-benefit patients actually have higher effects.)

What the Paper Did...

This is exactly the setting studied in:

Ferstad, Fox, Scheinker, Johari (2024). "Learning Explainable Treatment Policies with Clinician-Informed Representations." ML4H 2024. arXiv:2411.17570

Their key insight was that **the content of the message matters**.

The Role of Messages and NLP

- ▶ Clinicians send different *types* of messages: motivational, educational, reminders, check-ins.
- ▶ The **text of the message** is a rich source of information about the treatment.
- ▶ The authors use **NLP embeddings** (e.g., from language models) to represent message content as feature vectors.

Why NLP?

Rather than treating all messages as the same “treatment,” embeddings allow us to capture **what kind of intervention** was delivered. This turns the problem into a **multi-dimensional treatment** setting where the treatment is characterized by its embedding.

Clinician-Informed vs. Black-Box Representations

The paper makes a key distinction between two approaches to representing the data:

Black-Box Representations

- ▶ Use raw NLP embeddings of messages directly.
- ▶ High-dimensional, hard to interpret.
- ▶ May overfit with limited sample sizes.
- ▶ Clinicians can't understand or trust the policy.

Clinician-Informed Repr.

- ▶ Work with clinicians to define meaningful message categories.
- ▶ Lower-dimensional, interpretable.
- ▶ More robust with small samples.
- ▶ Clinicians can understand and act on the policy.

Main finding: Policies learned from **clinician-informed representations** are significantly more efficacious and efficient than those from black-box representations.

Connection to Today's Lecture

This case study connects directly to everything we've learned:

- ▶ **CATE estimation:** Estimate $\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x]$ for each patient—who benefits most from receiving a message?
- ▶ **Model selection:** The authors compare multiple CATE models. How would you use the DR-Loss to select among them?
- ▶ **Interpretation:** How would you explain the CATE to clinicians? Distillation trees and SHAP are ideal here.
- ▶ **Validation:** BLP test, GATEs, Qini curves—all applicable.
- ▶ **Observational data:** Since treatment is not randomized, we need good propensity and outcome models. The DR-Loss guarantee depends on nuisance quality.

Summary: The Complete CATE Workflow

1. **Train:** Fit multiple meta-learners (S, T, DR, R) with various ML methods. Tune hyperparameters using each learner's native loss.
2. **Select:** Use the DR-Loss on a held-out scoring set to select or stack the best CATE model. In RCTs, this is a perfect proxy for the true CATE MSE.
3. **Interpret:** Use distillation trees for the “big picture” and SHAP for granular, per-feature explanations.
4. **Validate:** On a held-out validation set, perform:
 - ▶ BLP test (is $\hat{\beta}$ non-zero?),
 - ▶ GATE analysis with calibration plot (are groups separated?),
 - ▶ Qini curve (is the model better than random for prioritization?).

You are now equipped to deploy reliable personalized decision-making systems!

References

- ▶ Chernozhukov, V., Demirer, M., Duflo, E., & Fern'andez-Val, I. (2023). Generic Machine Learning Inference on Heterogeneous Treatment Effects in Randomized Experiments. *Econometrica*, 91(1), 7–31.
- ▶ Nie, X., & Wager, S. (2021). Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika*, 108(2), 299–319.
- ▶ Foster, D. J., & Syrgkanis, V. (2023). Orthogonal statistical learning. *The Annals of Statistics*, 51(3), 879–908.
- ▶ Dwivedi, R., Tan, Y. S., Park, B., Wei, M., Horgan, K., Madigan, D., & Yu, B. (2020). Stable discovery of interpretable subgroups via calibration in causal studies. *International Statistical Review*, 88, S135-S178.
- ▶ Yadlowsky, S., Fleming, S., Shah, N., Brunskill, E., & Wager, S. (2025). Evaluating treatment prioritization rules via rank-weighted average treatment effects. *Journal of the American Statistical Association*, 120(549), 38-51.
- ▶ Ferstad, J. O., Fox, E. B., Scheinker, D., & Johari, R. (2024). Learning Explainable Treatment Policies with Clinician-Informed Representations. *arXiv:2411.17570*.